

Control de intentos biométricos — versión mínima

APP-5258 · Alcance recortado a lo estrictamente necesario · 25 de agosto de 2026
Microservicio afectado: `onboarding-micro-person` (el facade no requiere cambios de código) **Autor:** Deury Alcántara

1. Qué cubre esta versión

Sólo dos cosas: **el control de intentos en una colección de Mongo** y **lo que se envía hacia la tabla de auditoría**. El destino real de la auditoría todavía no se conoce, así que este documento deja el contrato y el payload cerrados, con un publicador provisional que escribe a logs; enchufar el destino real será una sola línea.

Todo lo demás del requerimiento que no sea imprescindible para que eso funcione queda fuera y está listado en la sección 11 para retomarlo después.

Total: 6 archivos nuevos, 4 modificados, cero cambios de código en `facade-security`.

	Versión completa	Versión mínima
Archivos nuevos en el micro	12	6
Archivos modificados	6	4
Cambios en <code>facade-security</code>	Swagger + 3 pruebas	Ninguno obligatorio
Pruebas unitarias	21	0 (ver §11)
Prerrequisito de cambio de contrato HTTP	Sí	No

2. Inventario de cambios

Archivos nuevos

Archivo	Ruta destino	Qué es
<code>FacePhiAttemptControlDocument.cs</code>	<code>Common/Entities/</code>	La entidad del estado
<code>IFacePhiAttemptControlRepository.cs</code>	<code>Common/Interfaces/Biometric/</code>	Contrato del repositorio
<code>FacePhiAttemptControlRepository.cs</code>	<code>Persistence/</code>	Operaciones atómicas contra Mongo
<code>BiometricAttemptsBlockedException.cs</code>	<code>Common/Exceptions/</code>	Se traduce a HTTP 423
<code>BiometricAttemptControlService.cs</code>	<code>Infrastructure/Biometric/</code>	Toda la política
<code>BiometricAudit.cs</code>	<code>Infrastructure/Biometric/</code>	Evento, contrato y publicador provisional

Archivos modificados

Archivo	Alcance del cambio
<code>Application/Facephi/Commands/ValidateIdentityV2Cmd.cs</code>	4 puntos en el handler
<code>Application/Facephi/Commands/ValidateFaceOnlyCmd.cs</code>	Los mismos 4 puntos
<code>Common/Configuration/AppSettings.cs</code> + <code>appsettings.json</code> + <code>ConfigMaps</code>	3 variables
Manejador de excepciones existente + <code>Program.cs</code>	Mapeo del 423 y 4 registros

`GetFacePhiDocumentQry` **no se toca**: consultar si hay documento almacenado no invoca a FacePhi, luego no es un intento y no debe contar ni bloquearse.

3. Qué se recortó y qué cuesta

Se quita	Coste real
<code>BiometricAttemptControlBehaviour</code> (pipeline de Mediatr)	Éste es el único coste de verdad. La regla "no se llama a FacePhi estando bloqueado" deja de ser estructural: hay que acordarse de replicar los cuatro puntos en el segundo handler, y un tercer flujo biométrico futuro no queda cubierto solo
<code>IBiometricAttemptControlled</code> (interfaz marcadora)	Ninguno: sólo existía para que el behaviour se enganchara
Enum <code>BiometricFlow</code> + <code>ToApiString()</code>	Ninguno: dos constantes <code>string</code> en el servicio hacen lo mismo
<code>TimeProvider</code> inyectable	Se pierde poder probar la expiración del bloqueo sin esperar 20 minutos. Se compensa forzando <code>blockedUntil</code> desde mongosh (\$10)
<code>FacePhiAttemptControlIndexInitializer</code> (<code>IHostedService</code>)	Ninguno, siempre que el índice se cree a mano en cada ambiente. No es opcional que exista el índice (\$5.3)
Índice <code>ix_isBlocked_blockedUntil</code>	Se pierde una consulta rápida de "cuántos clientes están bloqueados ahora". No afecta al funcionamiento
<code>BiometricAttemptContext</code>	Ninguno: el <code>requestId</code> se lee del <code>HttpContext</code> dentro del servicio
Las 21 pruebas unitarias	Se pierde la red de seguridad. Es lo primero que conviene recuperar (\$11)

4. El recorte que sale a favor

En la versión completa, el control vivía en un behaviour del pipeline. Eso obligaba a un prerequisite: distinguir "rechazo biométrico" de "fallo del servicio de FacePhi" **antes** de que el resultado saliera del handler. Como hoy ambos casos salen igual (`Status = Rejected`), había que lanzar una excepción de integración ante `serviceResultCode != 0` — y eso cambiaba el `200 { isValid: false }` por un `502` , con la coordinación con Móvil APAP que implica.

Invocando el control **desde dentro del handler**, ese problema desaparece. Ahí ya sabemos que `serviceResultCode != 0`, así que se decide en el momento:

```
if (result.ServiceResultCode != 0)
    await _attemptControl.RegisterTechnicalErrorAsync(...); // se audita, NO incrementa
else if (approved)
    await _attemptControl.RegisterSuccessAsync(...); // contador a 0
else
    await _attemptControl.RegisterRejectionAsync(...); // contador +1
```

Resultado: la app sigue recibiendo exactamente lo mismo que hoy, el contador queda correcto, y **el prerequisite sale del alcance**. Los mapeos pendientes de 400 y 502 siguen siendo deuda del desarrollo, pero ya no bloquean esta entrega.

El comportamiento importa: si un error técnico contara como intento fallido, **una caída de FacePhi bloquearía a todos los clientes que lo intentaran cinco veces**. Con esta separación, el contador sólo se mueve cuando FacePhi respondió correctamente y el veredicto fue del cliente.

5. Modelo de datos

5.1 Colección `facephi_attempt_control`

Nueva y **separada** de `facephi_biometrics`. Mezclarlas obligaría a escribir en el documento que guarda `token1` en cada intento fallido — justo el registro que no se quiere tocar.

```
{
  "_id": "66c9f1e2a4b3c2d1e0f9a8b7",
  "idT24": "123456789",
  "failedAttempts": 5,
  "isBlocked": true,
  "blockedAt": "2026-08-25T14:00:00Z",
  "blockedUntil": "2026-08-25T14:20:00Z",
  "lastFlow": "validate-face",
  "lastAttemptAt": "2026-08-25T14:00:00Z",
  "createdAt": "2026-08-20T09:11:03Z",
  "updatedAt": "2026-08-25T14:00:00Z"
}
```

Ningún campo biométrico entra aquí. Si alguien volcara la colección completa a un CSV, no habría fuga de datos biométricos.

5.2 Por qué el contador se incrementa con `$inc` y no con `ReplaceOneAsync`

Un contador de seguridad no se puede implementar leyendo-modificando-escribiendo. Dos peticiones concurrentes leen 4, escriben 5 las dos, y el cliente consigue un intento gratis en cada carrera. Todo el incremento ocurre del lado del servidor de Mongo, con `FindOneAndUpdateAsync` + `$inc`.

El bloqueo se aplica en una **segunda operación condicional** con filtro `isBlocked == false`: si dos peticiones cruzan el umbral a la vez, sólo una escribe la ventana y la otra no reinicia el reloj del bloqueo.

5.3 Índice único — no es opcional

En la versión mínima se crea a mano, una vez por ambiente:

```
db.facephi_attempt_control.createIndex({ idT24: 1 }, { unique: true, name: "ux_idT24" })
```

Sin él, dos peticiones concurrentes con upsert pueden insertar **dos documentos** para el mismo cliente, y a partir de ahí cada uno lleva su propio contador: el cliente tendría 10 intentos en lugar de 5.

Aprovechar el mismo cambio: `facephi_biometrics` sigue sin índice único sobre `idT24` y tiene exactamente la misma carrera en su `ReplaceOneAsync` con `IsUpsert`. Conviene crear los dos a la vez: `db.facephi_biometrics.createIndex({ idT24: 1 }, { unique: true, name: "ux_idT24" })`

6. Contrato del bloqueo (HTTP 423)

```
HTTP/1.1 423 Locked
Content-Type: application/problem+json
Retry-After: 1200
```

```
{
  "title": "Biometric validation temporarily blocked",
  "status": 423,
  "detail": "The customer exceeded the allowed number of failed biometric attempts.",
  "instance": "/api/v1/facephi/validate-face",
  "code": "BIOMETRIC_ATTEMPTS_BLOCKED",
  "isValid": false,
  "blocked": true,
  "blockedUntil": "2026-08-25T14:20:00Z",
  "retryAfterSeconds": 1200
}
```

Tres detalles deliberados:

- `isValid: false` **también en el 423**. La app lee siempre el mismo campo, responda `200` o `423`. No necesita dos rutas de parseo distintas.
- `retryAfterSeconds` **además de** `blockedUntil`. Un temporizador construido desde `blockedUntil` depende de que el reloj del teléfono esté sincronizado; con los segundos restantes, no depende de nada.
- **Cabecera** `Retry-After` **estándar**. Para que cualquier cliente HTTP genérico sepa comportarse sin conocer nuestro contrato.

Tabla completa de respuestas

Situación	Status	¿Consume intento?
Validación aprobada	200 { "isValid": true }	Sí — reinicia el contador a 0
Validación rechazada	200 { "isValid": false }	Sí — incrementa el contador
Petición inválida	400	No
Cliente sin documento almacenado	404	No
Cliente bloqueado	423	No — ni siquiera se llama a FacePhi
FacePhi respondió con serviceResultCode != 0	200 { "isValid": false }	No — se audita como error técnico
Excepción invocando FacePhi	500 (deuda pendiente: debería ser 502 / 504)	No — se audita como error técnico

7. Código

7.1 Entidad

Nuevo: `src/micro-person-api/Common/Entities/FacePhiAttemptControlDocument.cs`

```

// Ruta destino:
// src/micro-person-api/Common/Entities/FacePhiAttemptControlDocument.cs
//
// Colección Mongo NUEVA: "facephi_attempt_control".
// Separada de "facephi_biometrics" a propósito: aquella guarda token1/token2,
// y no queremos escribir en ese documento en cada intento fallido.
//
// Ningún campo biométrico entra aquí.

using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace onboarding_micro_person.Common.Entities
{
    public class FacePhiAttemptControlDocument
    {
        [BsonId]
        [BsonRepresentation(BsonType.ObjectId)]
        public string? Id { get; set; }

        [BsonElement("idT24")]
        public string IdT24 { get; set; } = string.Empty;

        /// <summary>Fallos consecutivos. Vuelve a 0 tras un éxito o al expirar el bloqueo.</summary>
        [BsonElement("failedAttempts")]
        public int FailedAttempts { get; set; }

        [BsonElement("isBlocked")]
        public bool IsBlocked { get; set; }

        [BsonElement("blockedAt")]
        public DateTime? BlockedAt { get; set; }

        [BsonElement("blockedUntil")]
        public DateTime? BlockedUntil { get; set; }

        /// <summary>"validate-biometric" o "validate-face".</summary>
        [BsonElement("lastFlow")]
        public string? LastFlow { get; set; }

        [BsonElement("lastAttemptAt")]
        public DateTime? LastAttemptAt { get; set; }

        [BsonElement("createdAt")]
        public DateTime CreatedAt { get; set; }

        [BsonElement("updatedAt")]
    }
}

```

```

public DateTime UpdatedAt { get; set; }

/// <summary>
/// El bloqueo está vigente sólo si la bandera está activa Y la ventana no ha
/// expirado. La bandera por sí sola no basta: nadie la apaga hasta el
/// siguiente intento.
/// </summary>
public bool IsCurrentlyBlocked(DateTime utcNow) =>
    IsBlocked && BlockedUntil is not null && BlockedUntil > utcNow;

public int RemainingSeconds(DateTime utcNow)
{
    if (BlockedUntil is null)
    {
        return 0;
    }

    var remaining = (BlockedUntil.Value - utcNow).TotalSeconds;

    return remaining <= 0 ? 0 : (int)Math.Ceiling(remaining);
}
}

```

7.2 Contrato del repositorio

Nuevo: `src/micro-person-api/Common/Interfaces/Biometric/IFacePhiAttemptControlRepository.cs`

```
// Ruta destino:
// src/micro-person-api/Common/Interfaces/Biometric/IFacePhiAttemptControlRepository.cs

using onboarding_micro_person.Common.Entities;

namespace onboarding_micro_person.Common.Interfaces.Biometric
{
    public interface IFacePhiAttemptControlRepository
    {
        Task<FacePhiAttemptControlDocument?> GetByIdT24Async(
            string idT24,
            Cancellation_token cancellation_token);

        /// <summary>Incrementa el contador de forma atómica y bloquea si alcanza el umbral.</summary>
        Task<FacePhiAttemptControlDocument> RegisterFailedAttemptAsync(
            string idT24,
            string flow,
            int maxFailedAttempts,
            int blockDurationMinutes,
            Cancellation_token cancellation_token);

        /// <summary>Reinicia el contador y limpia el bloqueo tras un intento exitoso.</summary>
        Task<FacePhiAttemptControlDocument> RegisterSuccessfulAttemptAsync(
            string idT24,
            string flow,
            Cancellation_token cancellation_token);

        /// <summary>
        /// Libera un bloqueo cuya ventana ya expiró y reinicia el contador.
        /// Condicional: devuelve null si otra petición simultánea se adelantó.
        /// </summary>
        Task<FacePhiAttemptControlDocument?> ReleaseExpiredBlockAsync(
            string idT24,
            Cancellation_token cancellation_token);
    }
}
```

7.3 Repositorio

Nuevo: src/micro-person-api/Persistence/FacePhiAttemptControlRepository.cs


```

// Ruta destino:
// src/micro-person-api/Persistence/FacePhiAttemptControlRepository.cs
//
// Ajusta la resolución de IMongoDatabase a como la hace FacePhiBiometricRepository.
//
// Este repositorio NO usa ReplaceOneAsync: un contador de seguridad se
// incrementa con $inc del lado del servidor, nunca leyendo-modificando-
// escribiendo desde la aplicación. Con read-modify-write, dos peticiones
// concurrentes leen 4, escriben 5 las dos, y el cliente consigue un intento
// gratis en cada carrera.

using MongoDB.Driver;
using onboarding_micro_person.Common.Entities;
using onboarding_micro_person.Common.Interfaces.Biometric;

namespace onboarding_micro_person.Persistence
{
    public class FacePhiAttemptControlRepository : IFacePhiAttemptControlRepository
    {
        public const string CollectionName = "facephi_attempt_control";

        private readonly IMongoCollection<FacePhiAttemptControlDocument> _collection;

        private static readonly FilterDefinitionBuilder<FacePhiAttemptControlDocument> Filter =
            Builders<FacePhiAttemptControlDocument>.Filter;

        private static readonly UpdateDefinitionBuilder<FacePhiAttemptControlDocument> Update =
            Builders<FacePhiAttemptControlDocument>.Update;

        public FacePhiAttemptControlRepository(IMongoDatabase database)
        {
            _collection = database.GetCollection<FacePhiAttemptControlDocument>(CollectionName);
        }

        public async Task<FacePhiAttemptControlDocument?> GetById24Async(
            string idT24,
            CancellationToken cancellationToken)
        {
            return await _collection
                .Find(d => d.IdT24 == idT24)
                .FirstOrDefaultAsync(cancellationToken);
        }

        public async Task<FacePhiAttemptControlDocument> RegisterFailedAttemptAsync(
            string idT24,
            string flow,
            int maxFailedAttempts,

```

```

    int blockDurationMinutes,
    CancellationToken cancellationToken)
{
    var utcNow = DateTime.UtcNow;

    // Paso 1: incremento atómico. El upsert crea el documento la primera vez;
    // el idT24 lo siembra Mongo a partir de la igualdad del filtro.
    var increment = Update
        .Inc(d => d.FailedAttempts, 1)
        .Set(d => d.LastFlow, flow)
        .Set(d => d.LastAttemptAt, utcNow)
        .Set(d => d.UpdatedAt, utcNow)
        .SetOnInsert(d => d.IsBlocked, false)
        .SetOnInsert(d => d.CreatedAt, utcNow);

    var state = await _collection.FindOneAndUpdateAsync(
        Filter.Eq(d => d.IdT24, idT24),
        increment,
        new FindOneAndUpdateOptions<FacePhiAttemptControlDocument>
        {
            IsUpsert = true,
            ReturnDocument = ReturnDocument.After
        },
        cancellationToken);

    if (state.FailedAttempts < maxFailedAttempts || state.IsBlocked)
    {
        return state;
    }

    // Paso 2: bloqueo condicional. El filtro exige isBlocked == false, así que
    // si dos peticiones cruzan el umbral a la vez sólo una escribe la ventana
    // y la otra no reinicia el reloj del bloqueo.
    var blockUpdate = Update
        .Set(d => d.IsBlocked, true)
        .Set(d => d.BlockedAt, utcNow)
        .Set(d => d.BlockedUntil, utcNow.AddMinutes(blockDurationMinutes))
        .Set(d => d.UpdatedAt, utcNow);

    var blocked = await _collection.FindOneAndUpdateAsync(
        Filter.And(
            Filter.Eq(d => d.IdT24, idT24),
            Filter.Eq(d => d.IsBlocked, false)),
        blockUpdate,
        new FindOneAndUpdateOptions<FacePhiAttemptControlDocument>
        {
            ReturnDocument = ReturnDocument.After
        },

```

```

        cancellationToken);

    return blocked ?? state;
}

public async Task<FacePhiAttemptControlDocument> RegisterSuccessfulAttemptAsync(
    string idT24,
    string flow,
    Cancellation token cancellationToken)
{
    var utcNow = DateTime.UtcNow;

    var update = Update
        .Set(d => d.FailedAttempts, 0)
        .Set(d => d.IsBlocked, false)
        .Set(d => d.BlockedAt, (DateTime?)null)
        .Set(d => d.BlockedUntil, (DateTime?)null)
        .Set(d => d.LastFlow, flow)
        .Set(d => d.LastAttemptAt, utcNow)
        .Set(d => d.UpdatedAt, utcNow)
        .SetOnInsert(d => d.CreatedAt, utcNow);

    return await _collection.FindOneAndUpdateAsync(
        Filter.Eq(d => d.IdT24, idT24),
        update,
        new FindOneAndUpdateOptions<FacePhiAttemptControlDocument>
        {
            IsUpsert = true,
            ReturnDocument = ReturnDocument.After
        },
        cancellationToken);
}

public async Task<FacePhiAttemptControlDocument?> ReleaseExpiredBlockAsync(
    string idT24,
    Cancellation token cancellationToken)
{
    var utcNow = DateTime.UtcNow;

    var update = Update
        .Set(d => d.IsBlocked, false)
        .Set(d => d.FailedAttempts, 0)
        .Set(d => d.BlockedAt, (DateTime?)null)
        .Set(d => d.BlockedUntil, (DateTime?)null)
        .Set(d => d.UpdatedAt, utcNow);

    return await _collection.FindOneAndUpdateAsync(
        Filter.And(

```

```

        Filter.Eq(d => d.IdT24, idT24),
        Filter.Eq(d => d.IsBlocked, true),
        Filter.Lte(d => d.BlockedUntil, utcNow)),
    update,
    new FindOneAndUpdateOptions<FacePhiAttemptControlDocument>
    {
        ReturnDocument = ReturnDocument.After
    },
    cancellationToken);
    }
}
}

```

7.4 Excepción de bloqueo

Nuevo: `src/micro-person-api/Common/Exceptions/BiometricAttemptsBlockedException.cs`

```
// Ruta destino:
// src/micro-person-api/Common/Exceptions/BiometricAttemptsBlockedException.cs
//
// Se traduce a HTTP 423 Locked. No hereda de ninguna excepción existente a
// propósito: confundirla con otra familia haría que el manejador genérico la
// convirtiera en 500.

namespace onboarding_micro_person.Common.Exceptions
{
    public class BiometricAttemptsBlockedException : Exception
    {
        public const string ErrorCode = "BIOMETRIC_ATTEMPTS_BLOCKED";

        public BiometricAttemptsBlockedException(
            string idT24,
            DateTime blockedUntil,
            int retryAfterSeconds)
            : base("Biometric validation is temporarily blocked for this customer.")
        {
            IdT24 = idT24;
            BlockedUntil = blockedUntil;
            RetryAfterSeconds = retryAfterSeconds;
        }

        public string IdT24 { get; }

        public DateTime BlockedUntil { get; }

        public int RetryAfterSeconds { get; }
    }
}
```

7.5 Auditoría

Nuevo: `src/micro-person-api/Infrastructure/Biometric/BiometricAudit.cs`

Ver la sección 9 para la explicación de cómo se enchufa el destino real.

```
// Ruta destino:
// src/micro-person-api/Infrastructure/Biometric/BiometricAudit.cs
//
// — ESTE ES EL PUNTO DE EXTENSIÓN PARA LA AUDITORÍA —
//
// Todavía no se sabe cuál es la base de datos ni el contrato de audit-log en
// micro-user-audit. Este archivo deja el desarrollo COMPLETO y funcionando sin
// esa información:
//
// · BiometricAuditEvent → el payload, ya definido y cerrado.
// · IBiometricAuditPublisher → el contrato, ya definido y cerrado.
// · LoggingBiometricAuditPublisher → implementación provisional que escribe
//   el evento en el log del micro como JSON.
//
// Cuando llegue el contrato real, se escribe UNA clase nueva
// (AuditLogBiometricAuditPublisher) que implemente la misma interfaz y se
// cambia UNA línea en Program.cs. Nada más del desarrollo se toca: ni el
// servicio, ni los handlers, ni el repositorio, ni las pruebas.
//
// Comandos para localizar la integración existente cuando toque:
// grep -rn "IAuditLog\|AuditService\|AuditClient\|audit-log" --include=*.cs src/
// grep -rni "audit" src/micro-person-api/appsettings*.json
// -----

using System.Text.Json;
using Microsoft.Extensions.Logging;

namespace onboarding_micro_person.Infrastructure.Biometric
{
    /// <summary>Catálogo cerrado de eventos auditables del control de intentos.</summary>
    public static class BiometricAuditEventType
    {
        public const string AttemptSuccess = "BIOMETRIC_ATTEMPT_SUCCESS";
        public const string AttemptFailed = "BIOMETRIC_ATTEMPT_FAILED";
        public const string BlockedByExcess = "BIOMETRIC_BLOCKED_BY_EXCESS";
        public const string AttemptWhileBlocked = "BIOMETRIC_ATTEMPT_WHILE_BLOCKED";
        public const string AutomaticUnblock = "BIOMETRIC_AUTOMATIC_UNBLOCK";
        public const string TechnicalError = "BIOMETRIC_TECHNICAL_ERROR";
    }

    /// <summary>
    /// Lo que se envía a la tabla de auditoría.
    ///
    /// Regla dura: aquí NO entra ningún dato biométrico. Nada de token1, token2,
    /// bestImageToken ni extraData. Se audita el hecho, no la evidencia.
    /// </summary>
    public class BiometricAuditEvent

```

```

{
    /// <summary>Uno de los valores de BiometricAuditEventType.</summary>
    public string EventType { get; set; } = string.Empty;

    /// <summary>Cliente.</summary>
    public string IdT24 { get; set; } = string.Empty;

    /// <summary>Fecha del evento, UTC.</summary>
    public DateTime OccurredAt { get; set; }

    /// <summary>"validate-biometric" o "validate-face".</summary>
    public string Flow { get; set; } = string.Empty;

    /// <summary>APPROVED | REJECTED | BLOCKED | UNBLOCKED | ERROR.</summary>
    public string Result { get; set; } = string.Empty;

    /// <summary>Contador de fallos consecutivos después de aplicar este evento.</summary>
    public int FailedAttempts { get; set; }

    /// <summary>Estado de bloqueo después de aplicar este evento.</summary>
    public bool Blocked { get; set; }

    public DateTime? BlockedUntil { get; set; }

    /// <summary>Trazabilidad: cabecera requestId de la petición.</summary>
    public string? RequestId { get; set; }

    /// <summary>Trazabilidad: serviceTransactionId devuelto por FacePhi.</summary>
    public string? ServiceTransactionId { get; set; }

    /// <summary>Sólo en TechnicalError: tipo de excepción o código de servicio. Sin stack trace.
    </summary>
    public string? ErrorDetail { get; set; }
}

public interface IBiometricAuditPublisher
{
    Task PublishAsync(BiometricAuditEvent auditEvent, CancellationToken cancellationToken);
}

/// <summary>
/// Implementación provisional: deja el evento en el log del micro, en JSON,
/// con un prefijo fijo para poder filtrarlo.
///
/// Sirve para que el desarrollo esté completo y verificable antes de conocer
/// el destino real, y para no perder los eventos de los ambientes de prueba
/// mientras tanto.
/// </summary>

```

```

public class LoggingBiometricAuditPublisher(
    ILogger<LoggingBiometricAuditPublisher> logger) : IBiometricAuditPublisher
{
    private static readonly JsonSerializerOptions SerializerOptions = new()
    {
        DefaultIgnoreCondition = System.Text.Json.Serialization.JsonIgnoreCondition.WhenWritingNull
    };

    private readonly ILogger<LoggingBiometricAuditPublisher> _logger = logger;

    public Task PublishAsync(BiometricAuditEvent auditEvent, CancellationToken cancellationToken)
    {
        _logger.LogInformation(
            "BIOMETRIC_AUDIT {AuditEvent}",
            JsonSerializer.Serialize(auditEvent, SerializerOptions));

        return Task.CompletedTask;
    }
}

// — PLANTILLA PARA LA IMPLEMENTACIÓN REAL —————
// Descomentar y ajustar cuando se conozca el contrato de audit-log.
// Al registrarla en Program.cs en lugar de LoggingBiometricAuditPublisher,
// el resto del desarrollo sigue igual.
//
// public class AuditLogBiometricAuditPublisher(
//     IAuditLogService auditLog,
//     ILogger<AuditLogBiometricAuditPublisher> logger) : IBiometricAuditPublisher
// {
//     public async Task PublishAsync(BiometricAuditEvent auditEvent, CancellationToken
cancellationToken)
//     {
//         try
//         {
//             await auditLog.RegisterAsync(new
//             {
//                 eventType = auditEvent.EventType,
//                 idT24 = auditEvent.IdT24,
//                 occurredAt = auditEvent.OccurredAt,
//                 flow = auditEvent.Flow,
//                 result = auditEvent.Result,
//                 failedAttempts = auditEvent.FailedAttempts,
//                 blocked = auditEvent.Blocked,
//                 blockedUntil = auditEvent.BlockedUntil,
//                 requestId = auditEvent.RequestId,
//                 serviceTransactionId = auditEvent.ServiceTransactionId,
//                 errorDetail = auditEvent.ErrorDetail
//             }, cancellationToken);

```



```

//      }
//      catch (Exception ex)
//      {
//          // La auditoría no puede tumbar la validación biométrica de un
//          // cliente. La fuente de verdad del contador es Mongo, no
//          // audit-log, así que el control sigue siendo correcto aunque se
//          // pierda un evento. El error queda en logs para poder alertarlo.
//          logger.LogError(
//              ex,
//              "Could not publish biometric audit event {EventType} | IdT24: {IdT24}",
//              auditEvent.EventType, auditEvent.IdT24);
//      }
//  }
// }
}

```

7.6 Servicio de control

Nuevo: `src/micro-person-api/Infrastructure/Biometric/BiometricAttemptControlService.cs`

```

// Ruta destino:
// src/micro-person-api/Infrastructure/Biometric/BiometricAttemptControlService.cs
//
// Toda la política vive aquí: umbral, ventana de bloqueo, desbloqueo automático
// y qué se audita. Los handlers sólo reportan el resultado de su intento.

using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.Logging;
using Microsoft.Extensions.Options;
using onboarding_micro_person.Common.Configuration;
using onboarding_micro_person.Common.Entities;
using onboarding_micro_person.Common.Exceptions;
using onboarding_micro_person.Common.Interfaces.Biometric;

namespace onboarding_micro_person.Infrastructure.Biometric
{
    public interface IBiometricAttemptControlService
    {
        /// <summary>
        /// Se llama ANTES de invocar a FacePhi. Si el cliente está bloqueado lanza
        /// BiometricAttemptsBlockedException (→ 423). Si el bloqueo ya expiró, lo
        /// libera y deja continuar.
        /// </summary>
        Task EnsureNotBlockedAsync(string idT24, string flow, CancellationToken cancellationToken);

        Task RegisterSuccessAsync(
            string idT24, string flow, string? serviceTransactionId,
            CancellationToken cancellationToken);

        Task RegisterRejectionAsync(
            string idT24, string flow, string? serviceTransactionId,
            CancellationToken cancellationToken);

        /// <summary>
        /// Error técnico: se audita pero NO incrementa el contador. Un cliente no
        /// puede quedar bloqueado porque FacePhi estuviera caído.
        /// </summary>
        Task RegisterTechnicalErrorAsync(
            string idT24, string flow, string errorDetail,
            CancellationToken cancellationToken);
    }

    public class BiometricAttemptControlService(
        IFacePhiAttemptControlRepository repository,
        IBiometricAuditPublisher auditPublisher,
        IHttpContextAccessor httpContextAccessor,
        IOptions<AppSettings> settings,

```

```

ILogger<BiometricAttemptControlService> logger) : IBiometricAttemptControlService
{
    public const string FlowValidateBiometric = "validate-biometric";
    public const string FlowValidateFace = "validate-face";

    private readonly IFacePhiAttemptControlRepository _repository = repository;
    private readonly IBiometricAuditPublisher _auditPublisher = auditPublisher;
    private readonly IHttpContextAccessor _httpContextAccessor = httpContextAccessor;
    private readonly AppSettings _settings = settings.Value;
    private readonly ILogger<BiometricAttemptControlService> _logger = logger;

    public async Task EnsureNotBlockedAsync(
        string idT24,
        string flow,
        CancellationToken cancellationToken)
    {
        if (!_settings.BIOMETRIC_ATTEMPT_CONTROL_ENABLED)
        {
            return;
        }

        var utcNow = DateTime.UtcNow;
        var state = await _repository.GetByIdT24Async(idT24, cancellationToken);

        if (state is null || !state.IsBlocked)
        {
            return;
        }

        // Desbloqueo automático perezoso: el bloqueo sólo tiene efecto cuando
        // alguien intenta validar, así que es aquí donde se comprueba si la
        // ventana venció. No hace falta un job programado.
        if (!state.IsCurrentlyBlocked(utcNow))
        {
            var released = await _repository.ReleaseExpiredBlockAsync(idT24, cancellationToken);

            // released == null significa que otra petición simultánea ya lo
            // liberó. El resultado es el mismo: el cliente puede continuar.
            if (released is not null)
            {
                _logger.LogInformation(
                    "Biometric block expired and was released | IdT24: {IdT24} | Flow: {Flow}",
                    idT24, flow);

                await PublishAsync(
                    BiometricAuditEventType.AutomaticUnblock, idT24, flow, utcNow,
                    "UNBLOCKED", released, null, null, cancellationToken);
            }
        }
    }
}

```

```

        return;
    }

    var retryAfterSeconds = state.RemainingSeconds(utcNow);

    _logger.LogWarning(
        "Biometric validation attempted while blocked | IdT24: {IdT24} | Flow: {Flow} | " +
        "BlockedUntil: {BlockedUntil} | RetryAfterSeconds: {RetryAfterSeconds}",
        idT24, flow, state.BlockedUntil, retryAfterSeconds);

    await PublishAsync(
        BiometricAuditEventType.AttemptWhileBlocked, idT24, flow, utcNow,
        "BLOCKED", state, null, null, cancellationTokens);

    throw new BiometricAttemptsBlockedException(
        idT24, state.BlockedUntil!.Value, retryAfterSeconds);
}

public async Task RegisterSuccessAsync(
    string idT24,
    string flow,
    string? serviceTransactionId,
    CancellationToken cancellationToken)
{
    var utcNow = DateTime.UtcNow;

    FacePhiAttemptControlDocument? state = null;

    if (_settings.BIOMETRIC_ATTEMPT_CONTROL_ENABLED)
    {
        state = await _repository.RegisterSuccessfulAttemptAsync(idT24, flow, cancellationToken);

        _logger.LogInformation(
            "Biometric attempt counter reset after success | IdT24: {IdT24} | Flow: {Flow}",
            idT24, flow);
    }

    await PublishAsync(
        BiometricAuditEventType.AttemptSuccess, idT24, flow, utcNow,
        "APPROVED", state, serviceTransactionId, null, cancellationTokens);
}

public async Task RegisterRejectionAsync(
    string idT24,
    string flow,
    string? serviceTransactionId,
    CancellationToken cancellationToken)

```

```

{
    var utcNow = DateTime.UtcNow;

    if (!_settings.BIOMETRIC_ATTEMPT_CONTROL_ENABLED)
    {
        // Con el control apagado se sigue auditando: se pierde el bloqueo,
        // no la trazabilidad.
        await PublishAsync(
            BiometricAuditEventType.AttemptFailed, idT24, flow, utcNow,
            "REJECTED", null, serviceTransactionId, null, cancellationToken);

        return;
    }

    var state = await _repository.RegisterFailedAttemptAsync(
        idT24,
        flow,
        _settings.BIOMETRIC_MAX_FAILED_ATTEMPTS,
        _settings.BIOMETRIC_BLOCK_DURATION_MINUTES,
        cancellationToken);

    _logger.LogWarning(
        "Biometric attempt rejected | IdT24: {IdT24} | Flow: {Flow} | " +
        "FailedAttempts: {FailedAttempts} | Blocked: {Blocked}",
        idT24, flow, state.FailedAttempts, state.IsBlocked);

    await PublishAsync(
        BiometricAuditEventType.AttemptFailed, idT24, flow, utcNow,
        "REJECTED", state, serviceTransactionId, null, cancellationToken);

    // El bloqueo es un evento distinto del fallo que lo provoca: soporte
    // necesita poder responder "¿cuándo se bloqueó?" sin recomponer la
    // secuencia de cinco fallos.
    if (state.IsCurrentlyBlocked(utcNow))
    {
        _logger.LogWarning(
            "Customer blocked for biometric validation | IdT24: {IdT24} | " +
            "FailedAttempts: {FailedAttempts} | BlockedUntil: {BlockedUntil}",
            idT24, state.FailedAttempts, state.BlockedUntil);

        await PublishAsync(
            BiometricAuditEventType.BlockedByExcess, idT24, flow, utcNow,
            "BLOCKED", state, serviceTransactionId, null, cancellationToken);
    }
}

public async Task RegisterTechnicalErrorAsync(
    string idT24,

```

```

        string flow,
        string errorDetail,
        CancellationToken cancellationToken)
    {
        var utcNow = DateTime.UtcNow;

        // Deliberadamente NO se llama a RegisterFailedAttemptAsync.
        // Un error técnico no es un intento fallido del cliente.
        var state = await _repository.GetByIdT24Async(idT24, cancellationToken);

        _logger.LogError(
            "Biometric technical error, counter not incremented | IdT24: {IdT24} | " +
            "Flow: {Flow} | Detail: {Detail}",
            idT24, flow, errorDetail);

        await PublishAsync(
            BiometricAuditEventType.TechnicalError, idT24, flow, utcNow,
            "ERROR", state, null, errorDetail, cancellationToken);
    }

    private Task PublishAsync(
        string eventType,
        string idT24,
        string flow,
        DateTime utcNow,
        string result,
        FacePhiAttemptControlDocument? state,
        string? serviceTransactionId,
        string? errorDetail,
        CancellationToken cancellationToken)
    {
        return _auditPublisher.PublishAsync(
            new BiometricAuditEvent
            {
                EventType = eventType,
                IdT24 = idT24,
                Flow = flow,
                OccurredAt = utcNow,
                Result = result,
                FailedAttempts = state?.FailedAttempts ?? 0,
                Blocked = state?.IsCurrentlyBlocked(utcNow) ?? false,
                BlockedUntil = state?.BlockedUntil,
                RequestId = _httpContextAccessor.HttpContext?
                    .Request.Headers["requestId"].FirstOrDefault(),
                ServiceTransactionId = serviceTransactionId,
                ErrorDetail = errorDetail
            },
            cancellationToken);
    }

```

```
}  
}  
}
```

8. Cambios en los handlers

```
// -----  
// CAMBIOS EN LOS DOS HANDLERS EXISTENTES  
//  
// En la versión mínima el control se invoca directamente desde el handler, sin  
// behaviour de MediatR. Se pierde el que la regla sea estructural (hay que  
// acordarse de replicar los cuatro puntos en el segundo handler), pero se gana  
// algo importante:  
//  
// Dentro del handler ya sabemos si serviceResultCode != 0, así que podemos  
// distinguir "rechazo biométrico" de "fallo del servicio de FacePhi" SIN  
// cambiar el contrato HTTP. La app sigue recibiendo 200 { isValid: false }  
// igual que hoy, y el contador no se mueve por una caída de FacePhi.  
//  
// Son cuatro puntos por handler, marcados abajo como (1) a (4).  
// -----  
  
// =====  
// A) ValidateFaceOnlyCmdHandler - handler completo, ya modificado  
// src/micro-person-api/Application/FacePhi/Commands/ValidateFaceOnlyCmd.cs  
// =====  
  
public class ValidateFaceOnlyCmdHandler(  
    IFacePhiService facePhi,  
    IFacePhiBiometricRepository repository,  
    IIdentityValidationEvaluator identityValidationEvaluator,  
    IBiometricAttemptControlService attemptControl, // ← (1) nueva dependencia  
    IOptions<AppSettings> settings,  
    ILogger<ValidateFaceOnlyCmdHandler> logger  
) : IRequestHandler<ValidateFaceOnlyCmd, ValidateIdentityV2Result>  
{  
    private const int FacialAuthenticationPositive = 3;  
  
    private const string Flow = BiometricAttemptControlService.FlowValidateFace;  
  
    private readonly IFacePhiService _facePhi = facePhi;  
    private readonly IFacePhiBiometricRepository _repository = repository;  
    private readonly IIdentityValidationEvaluator _identityValidationEvaluator =  
identityValidationEvaluator;  
    private readonly IBiometricAttemptControlService _attemptControl = attemptControl; // ← (1)  
    private readonly AppSettings _settings = settings.Value;  
    private readonly ILogger<ValidateFaceOnlyCmdHandler> _logger = logger;  
  
    public async Task<ValidateIdentityV2Result> Handle(  
        ValidateFaceOnlyCmd cmd,
```



```

CancellationToken cancellationToken)
{
    _logger.LogInformation("Starting face-only validation for IdT24: {IdT24}", cmd.IdT24);

    // — (2) PUERTA DE ENTRADA —————
    // Va antes de todo lo demás. Si el cliente está bloqueado, lanza
    // BiometricAttemptsBlockedException y no se ejecuta ni la consulta a
    // Mongo ni la llamada a FacePhi.
    await _attemptControl.EnsureNotBlockedAsync(cmd.IdT24, Flow, cancellationToken);

    var biometric = await _repository.GetByIdT24Async(cmd.IdT24, cancellationToken);

    // No basta con que exista el registro: tiene que tener el token del documento.
    // Esto NO consume intento: FacePhi todavía no se ha invocado.
    if (string.IsNullOrEmpty(biometric?.Token1))
    {
        _logger.LogWarning("No stored document found | IdT24: {IdT24}", cmd.IdT24);

        throw new NotFoundException("Customer does not have a stored document.");
    }

    // — (3) LA LLAMADA A FACEPHI, ENVUELTA —————
    // Si revienta (red, timeout, 5xx), se audita como error técnico y se
    // relanza. El contador no se toca: no fue culpa del cliente.
    PassiveLivenessResponse result;

    try
    {
        result = await _facePhi.EvaluatePassiveLivenessToken(
            new PassiveLivenessRequest
            {
                Token1 = biometric.Token1,
                BestImageToken = cmd.BestImageToken,
                Method = biometric.Method.ToString(),
                TrackingToken = cmd.Tracking?.ExtraData,
                OperationId = cmd.Tracking?.OperationId
            },
            cancellationToken);
    }
    catch (Exception ex)
    {
        await _attemptControl.RegisterTechnicalErrorAsync(
            cmd.IdT24, Flow, ex.GetType().Name, cancellationToken);

        throw;
    }

    _logger.LogInformation(

```

```

        "LivenessLog: {LivenessLog} | AuthenticationLog: {AuthenticationLog} | IdT24: {IdT24}",
        result.passiveLivenessLog, result.facialAuthenticationLog, cmd.IdT24);

var approved = false;
var similarity = result.facialAuthenticationSimilarity;

if (result.ServiceResultCode != 0)
{
    _logger.LogWarning(
        "Unsuccessful service result | ServiceResultCode: {ServiceResultCode} | " +
        "TransactionId: {TransactionId} | IdT24: {IdT24}",
        result.ServiceResultCode, result.ServiceTransactionId, cmd.IdT24);
}
else
{
    // La prueba de vida es el control principal de este flujo:
    // el cliente ya no presenta el documento físico, sólo su rostro.
    var isAlive = result.passiveLivenessResult == FacephilivenessResult.Live;

    if (_settings.FACIAL_VALIDATION_CUSTOM_LIVENESS_CHECK)
    {
        var meetsSimilarityThreshold =
            _identityValidationEvaluator.MeetsSimilarityThreshold(similarity);

        approved = meetsSimilarityThreshold && isAlive;

        _logger.LogInformation(
            "Custom liveness check result | Similarity: {Similarity:F4} | " +
            "MeetsSimilarityThreshold: {MeetsSimilarityThreshold} | IsAlive: {IsAlive} | IdT24:
{IdT24}",
            similarity, meetsSimilarityThreshold, isAlive, cmd.IdT24);
    }
    else
    {
        approved = result.facialAuthenticationResult == FacialAuthenticationPositive && isAlive;
    }
}

var validationStatus = approved
    ? FacialValidationStatus.Approved
    : FacialValidationStatus.Rejected;

_logger.LogInformation(
    "Face-only validation result: {Status} | Similarity: {Similarity:F4} | IdT24: {IdT24}",
    validationStatus, similarity, cmd.IdT24);

// — (4) REGISTRO DEL RESULTADO —————
// Aquí está la clave de por qué la versión mínima no necesita cambiar el

```

```

// contrato HTTP: dentro del handler sabemos que serviceResultCode != 0
// significa fallo del servicio, no rechazo del cliente. La app sigue
// recibiendo 200 { isValid: false }, pero el contador no se mueve.
if (result.ServiceResultCode != 0)
{
    await _attemptControl.RegisterTechnicalErrorAsync(
        cmd.IdT24,
        Flow,
        $"serviceResultCode={result.ServiceResultCode}",
        cancellationToken);
}
else if (approved)
{
    await _attemptControl.RegisterSuccessAsync(
        cmd.IdT24, Flow, result.ServiceTransactionId, cancellationToken);
}
else
{
    await _attemptControl.RegisterRejectionAsync(
        cmd.IdT24, Flow, result.ServiceTransactionId, cancellationToken);
}

var trackingToken = cmd.Tracking?.ExtraData;

if (!string.IsNullOrEmpty(trackingToken))
{
    var reason = (approved
        ? FacePhiTrackingReason.None
        : FacePhiTrackingReason.FacialAuthenticationNotPassed).ToApiString();

    await _facePhi.FinishTrackingAsync(approved, reason, trackingToken, cancellationToken);
}

_logger.LogInformation(
    "Face-only validation completed for IdT24: {IdT24} with Status: {Status} " +
    "and TransactionId: {TransactionId}",
    cmd.IdT24, validationStatus, result.ServiceTransactionId);

return new ValidateIdentityV2Result(validationStatus, result);
}
}

// =====
// B) ValidateIdentityV2CmdHandler - exactamente los mismos cuatro puntos
// src/micro-person-api/Application/Facephi/Commands/ValidateIdentityV2Cmd.cs
// =====
//

```

```

// (1) Añadir al constructor primario y al campo:
//
//         IBiometricAttemptControlService attemptControl,
//         ...
//         private readonly IBiometricAttemptControlService _attemptControl = attemptControl;
//
// Y la constante del flujo:
//
//         private const string Flow = BiometricAttemptControlService.FlowValidateBiometric;
//
// (2) Como PRIMERA línea del Handle, antes de cualquier otra cosa:
//
//         await _attemptControl.EnsureNotBlockedAsync(cmd.IdT24, Flow, cancellationToken);
//
// (3) Envolver la llamada a FacePhi en el mismo try/catch:
//
//         try
//         {
//             result = await _facePhi.EvaluatePassiveLivenessToken(...);
//         }
//         catch (Exception ex)
//         {
//             await _attemptControl.RegisterTechnicalErrorAsync(
//                 cmd.IdT24, Flow, ex.GetType().Name, cancellationToken);
//             throw;
//         }
//
// (4) Después de calcular validationResult y ANTES del FinishTrackingAsync,
//     el mismo bloque de tres ramas:
//
//         if (result.ServiceResultCode != 0)
//             await _attemptControl.RegisterTechnicalErrorAsync(
//                 cmd.IdT24, Flow, $"serviceResultCode={result.ServiceResultCode}", cancellationToken);
//         else if (approved)
//             await _attemptControl.RegisterSuccessAsync(
//                 cmd.IdT24, Flow, result.ServiceTransactionId, cancellationToken);
//         else
//             await _attemptControl.RegisterRejectionAsync(
//                 cmd.IdT24, Flow, result.ServiceTransactionId, cancellationToken);
//
//
// NOTA sobre el tipo de "result": arriba se declaró como PassiveLivenessResponse
// para poder sacarlo del try. Sustitúyelo por el tipo real que devuelve
// EvaluatePassiveLivenessToken en el micro (míralo en IFacePhiService).
//
// NOTA sobre GetFacePhiDocumentQry: NO se toca. Consultar si hay documento

```

```
// almacenado no invoca a FacePhi, luego no es un intento y no debe contar ni
// bloquearse.
```

9. La parte de auditoría

El requerimiento pide usar la integración existente con `audit-log` en `micro-user-audit`, pero todavía no se conoce ni la base de datos ni el contrato. El desarrollo **no se bloquea por eso**.

9.1 Lo que queda cerrado desde ya

Pieza	Estado
<code>BiometricAuditEvent</code> — el payload	Definido y cerrado
<code>IBiometricAuditPublisher</code> — el contrato	Definido y cerrado
Catálogo de eventos	Definido y cerrado (6 tipos)
<code>LoggingBiometricAuditPublisher</code>	Implementación provisional: escribe el evento en JSON al log del micro con prefijo <code>BIOMETRIC_AUDIT</code>

9.2 Catálogo de eventos

Evento	Cuándo se emite	result	¿Contador se mueve?
<code>BIOMETRIC_ATTEMPT_SUCCESS</code>	Validación aprobada	<code>APPROVED</code>	Se reinicia a 0
<code>BIOMETRIC_ATTEMPT_FAILED</code>	Validación rechazada por FacePhi	<code>REJECTED</code>	+1
<code>BIOMETRIC_BLOCKED_BY_EXCESS</code>	El fallo alcanzó el umbral	<code>BLOCKED</code>	No (lo movió el evento anterior)
<code>BIOMETRIC_ATTEMPT_WHILE_BLOCKED</code>	Llamada recibida durante el bloqueo	<code>BLOCKED</code>	No
<code>BIOMETRIC_AUTOMATIC_UNBLOCK</code>	La ventana expiró y se liberó	<code>UNBLOCKED</code>	Se reinicia a 0
<code>BIOMETRIC_TECHNICAL_ERROR</code>	Fallo de servicio o excepción invocando FacePhi	<code>ERROR</code>	No

El bloqueo se emite como evento **separado** del fallo que lo provoca: soporte necesita poder responder "¿cuándo se bloqueó este cliente?" consultando un evento, no recomponiendo la secuencia de cinco fallos.

9.3 Campos del payload

Campo	Origen
eventType	Catálogo de 9.2
idT24	Comando
occurredAt	UTC
flow	validate-biometric validate-face
result	APPROVED REJECTED BLOCKED UNBLOCKED ERROR
failedAttempts	Estado tras aplicar el evento
blocked	Estado tras aplicar el evento
blockedUntil	Estado, sólo si está bloqueado
requestId	Cabecera HTTP (trazabilidad)
serviceTransactionId	Respuesta de FacePhi (trazabilidad)
errorDetail	Sólo en TECHNICAL_ERROR : tipo de excepción o código de servicio, sin stack trace

Lo que nunca entra: token1 · token2 · bestImageToken · extraData · imágenes. La auditoría registra **el hecho**, no la evidencia.

9.4 Cómo se enchufa el destino real

Cuando se conozca el contrato de audit-log :

1. Localizar el cliente existente: `bash grep -rn "IAuditLog\|AuditService\|AuditClient\|audit-log" --include=*.cs src/ grep -rni "audit" src/micro-person-api/appsettings*.json`
2. Escribir `AuditLogBiometricAuditPublisher` implementando `IBiometricAuditPublisher` — la plantilla está comentada al final de `BiometricAudit.cs`.
3. Cambiar **una línea** en `Program.cs` : `csharp builder.Services.AddScoped<IBiometricAuditPublisher, AuditLogBiometricAuditPublisher>();`

Ni el servicio, ni los handlers, ni el repositorio se tocan.

9.5 La auditoría no puede tumbar el flujo

La implementación real debe tragar sus propios errores y dejar constancia en logs. Si `audit-log` está caído, el cliente sigue pudiendo validar: la fuente de verdad del contador es Mongo, no la auditoría, así que el control sigue siendo correcto aunque se pierda un evento. La alternativa —fallar la validación biométrica de un cliente porque el servicio de auditoría no responde— cambia una pérdida de trazabilidad por una caída de un flujo crítico de cara al cliente.

Recomendación operativa: alertar sobre el log `Could not publish biometric audit event`. La pérdida de eventos tiene que ser detectable, no silenciosa.

10. Registros, configuración, 423 e índice

VERSIÓN MÍNIMA · Registros, configuración, mapeo del 423 e índice

1. Program.cs / DependencyInjection.cs

Sólo se añaden líneas. No se reemplaza nada. No se toca el pipeline de MediatR.

```
// El servicio lee el requestId de la cabecera para la trazabilidad de auditoría.
builder.Services.AddHttpContextAccessor();

builder.Services.AddScoped<IFacePhiAttemptControlRepository, FacePhiAttemptControlRepository>();
builder.Services.AddScoped<IBiometricAttemptControlService, BiometricAttemptControlService>();

// Auditoría: implementación provisional mientras no se conozca el destino real.
// Cuando llegue el contrato de audit-log, se cambia SÓLO esta línea por:
//     builder.Services.AddScoped<IBiometricAuditPublisher, AuditLogBiometricAuditPublisher>();
builder.Services.AddScoped<IBiometricAuditPublisher, LoggingBiometricAuditPublisher>();
```

2. AppSettings.cs

Tres propiedades nuevas:

```
/// <summary>Interruptor general. En false no se bloquea a nadie, pero se sigue auditando.</summary>
public bool BIOMETRIC_ATTEMPT_CONTROL_ENABLED { get; set; } = true;

/// <summary>Fallos consecutivos que disparan el bloqueo.</summary>
public int BIOMETRIC_MAX_FAILED_ATTEMPTS { get; set; } = 5;

/// <summary>Duración del bloqueo temporal, en minutos.</summary>
public int BIOMETRIC_BLOCK_DURATION_MINUTES { get; set; } = 20;
```

El interruptor no es decoración: si en producción aparece un falso positivo masivo (por ejemplo, una versión de la app que envía capturas de mala calidad y dispara rechazos legítimos), se apaga el bloqueo SIN DESPLEGAR y la auditoría sigue funcionando.

3. appsettings.json · dentro de la sección AppSettings existente

```
"BIOMETRIC_ATTEMPT_CONTROL_ENABLED": true,
"BIOMETRIC_MAX_FAILED_ATTEMPTS": 5,
```

```
"BIOMETRIC_BLOCK_DURATION_MINUTES": 20
```

4. ConfigMap de Kubernetes · los tres ambientes

```
AppSettings__BIOMETRIC_ATTEMPT_CONTROL_ENABLED: "true"
AppSettings__BIOMETRIC_MAX_FAILED_ATTEMPTS: "5"
AppSettings__BIOMETRIC_BLOCK_DURATION_MINUTES: "20"
```

5. Mapeo del 423 en el manejador de excepciones que YA existe

Localízalo con:

```
grep -rn "ExceptionHandler\|ProblemDetails\|ExceptionMiddleware" --include=*.cs src/
```

Si es un CustomExceptionHandler con diccionario de mapeos (patrón de la plantilla Clean Architecture), añade la entrada y el método:

```
{ typeof(BiometricAttemptsBlockedException), HandleAttemptsBlockedException }

private async Task HandleAttemptsBlockedException(HttpContext httpContext, Exception ex)
{
    var blocked = (BiometricAttemptsBlockedException)ex;

    httpContext.Response.StatusCode = StatusCodes.Status423Locked;
    httpContext.Response.Headers.RetryAfter = blocked.RetryAfterSeconds.ToString();

    var problemDetails = new ProblemDetails
    {
        Status = StatusCodes.Status423Locked,
        Title = "Biometric validation temporarily blocked",
        Detail = "The customer exceeded the allowed number of failed biometric attempts.",
        Instance = httpContext.Request.Path
    };

    problemDetails.Extensions["code"] = BiometricAttemptsBlockedException.ErrorCode;
    problemDetails.Extensions["isValid"] = false;           // la app lee siempre el mismo campo
    problemDetails.Extensions["blocked"] = true;
    problemDetails.Extensions["blockedUntil"] = blocked.BlockedUntil;
    problemDetails.Extensions["retryAfterSeconds"] = blocked.RetryAfterSeconds;

    await httpContext.Response.WriteAsJsonAsync(problemDetails);
}
```

Tres detalles deliberados:

- isValid: false también en el 423 → la app lee siempre el mismo campo, responda 200 o 423. No necesita dos rutas de parseo.
- retryAfterSeconds además de blockedUntil → un temporizador hecho con blockedUntil depende de que el reloj del teléfono esté sincronizado.
- Cabecera Retry-After estándar → para cualquier cliente HTTP genérico.

6. Índice único en Mongo

En la versión mínima se crea a mano una vez por ambiente, en lugar de añadir un IHostedService:

```
db.facephi_attempt_control.createIndex({ idT24: 1 }, { unique: true, name: "ux_idT24" })
```

N0 es opcional. Sin él, dos peticiones concurrentes con upsert pueden insertar DOS documentos para el mismo cliente, y a partir de ahí cada uno lleva su propio contador: el cliente tendría 10 intentos en lugar de 5.

Aprovechando el mismo cambio, conviene crear también el que sigue pendiente en la otra colección, que tiene exactamente la misma carrera en su ReplaceOneAsync con IsUpsert:

```
db.facephi_biometrics.createIndex({ idT24: 1 }, { unique: true, name: "ux_idT24" })
```

7. facade-security

CERO cambios de código. handleUpstreamError ya reenvía error.response.status tal cual, así que el 423 y su cuerpo pasan íntegros hasta la app.

Lo único recomendable (y opcional en la versión mínima) es documentarlo en Swagger, en validateBiometric y validateFaceOnly:

```
@ApiResponse({
  status: 423,
  description:
    'Validacion biometrica bloqueada temporalmente por exceso de intentos fallidos.',
})
```

8. Verificación antes de cerrar

Confirmar que NADA entre el micro y la app remapea el 423: filtros globales del facade, middleware de cifrado, APIM.

La forma rápida de cerrarlo: fallar cinco veces la validación en un ambiente de pruebas y observar el status QUE RECIBE LA APP, no el que emite el micro.

```
for i in 1 2 3 4 5 6; do
  curl -s -o /dev/null -w "intento $i -> %{http_code}\n" \
    -X POST http://localhost:5000/api/v1/facephi/validate-face \
    -H "Content-Type: application/json" -H "requestId: manual-$i" \
    -d '{"idT24": "999999999", "bestImageToken": "<token>"}'
done

# esperado: 200 200 200 200 200 423

db.facephi_attempt_control.findOne({ idT24: "999999999" })
# esperado: failedAttempts: 5, isBlocked: true, blockedUntil ≈ ahora + 20 min

# Forzar la expiración sin esperar 20 minutos:
db.facephi_attempt_control.updateOne(
  { idT24: "999999999" },
  { $set: { blockedUntil: new Date(Date.now() - 1000) } })
# el siguiente intento debe devolver 200 y dejar failedAttempts en 0
```

Y para los eventos de auditoría, mientras el publicador sea el provisional:

```
kubectl logs <pod> | grep BIOMETRIC_AUDIT
```

11. Lo que queda fuera y cuándo retomarlo

Pendiente	Prioridad	Por qué
Las pruebas unitarias	Alta — retomar primero	Sin ellas, nada verifica que el quinto fallo bloquee, que un éxito reinicie o que un error técnico no incremente. La versión completa trae 21, con un repositorio en memoria que reproduce la semántica de Mongo
Publicador real de <code>audit-log</code>	Alta — en cuanto llegue el contrato	Hasta entonces los eventos viven en los logs del micro. Es una línea de cambio (\$9.4)
<code>BiometricAttemptControlBehaviour</code>	Media	Convierte "no se llama a FacePhi estando bloqueado" en garantía estructural en lugar de convención. Requiere el prerequisite del 502 (\$4)
Mapeos 400 / 502 / 504	Media	Deuda anterior a este desarrollo: hoy <code>ValidationException</code> y los fallos de FacePhi salen como 500
<code>IHostedService</code> de índices	Baja	Sólo si molesta crear el índice a mano en cada ambiente
Swagger 423 en el facade	Baja	No afecta al funcionamiento; el 423 ya se propaga solo
Prueba de integración con Mongo real	Baja	Verificaría la atomicidad del <code>\$inc</code> bajo 20 peticiones concurrentes y que el índice único impida duplicados

12. Verificación antes de cerrar

Dos cosas que no conviene dar por hechas.

El índice existe en el ambiente.

```
db.facephi_attempt_control.getIndexes()  
// debe aparecer ux_idT24 con unique: true
```

Nada remapea el 423 entre el micro y la app. Filtros globales del facade, middleware de cifrado, APIM. La forma rápida de cerrarlo es fallar cinco veces la validación en un ambiente de pruebas y observar el status **que recibe la app**, no el que emite el micro. Los comandos están en la sección 10, punto 8.